certora

Formal Verification
Draft Report

Royco

Royco Dawn

May 2026

Prepared for Royco

certora

Royco

Table of Contents

certora Royco

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Royco Dawn	https://github.com/roycoprotocol/royco-dawn	09ac793d9b5b27de7cc6d8e5a0e5649a58437bf2	EVM

Project Overview

This document describes the specification and verification of **Royco Dawn** using the Certora Prover and manual code review findings. The work was undertaken from **April 15** to **May 31, 2026.**

The following contract list is included in our scope:

```
src/accountant/RoycoAccountant.sol
src/kernels/base/RoycoKernel.sol
src/kernels/Identical_ERC20_ST_JT_ChainlinkToAdminOracle_Kernel.sol
src/kernels/Identical_ERC4626_ST_JT_ChainlinkToAdminOracle_Kernel.sol
src/tranches/base/RoycoVaultTranche.sol
src/tranches/RoycoSeniorTranche.sol
src/tranches/RoycoJuniorTranche.sol
```

The Certora Prover demonstrated that the implementation of the **Solidity** contracts above is correct with respect to the formal rules written by the Certora team**.** During the verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following page.

Protocol Overview

Royco Dawn is a non-custodial risk-tranching protocol that takes a yield source, such as a lending market, staking deposit, or tokenized real-world asset, and splits it into two distinct positions: a Senior tranche, which earns yield with smart-contract-enforced downside protection, and a Junior tranche, which absorbs losses first in exchange for higher yield and a risk premium paid by Senior depositors. The yield split between the two tranches continuously adapts to supply and demand signals from each side, allowing the market to price each opportunity in real time. In essence, it brings traditional credit waterfall structuring, which is common in institutional finance, to DeFi, letting participants choose their preferred risk profile.

Threat and Security Overview

Royco Dawn is a structured-finance vault protocol that splits depositor capital into two risk tranches: a senior tranche (ST) that receives downside protection up to a configurable coverage ratio, and a junior tranche (JT) that absorbs losses first in exchange for a larger share of yield. The core accounting logic lives in the RoycoAccountant, which tracks raw and effective NAV for each tranche, manages impermanent loss state, and enforces coverage constraints on every operation. The RoycoKernel holds the underlying assets, manages the configured price oracle used to value tokens, and exposes tranche-only ST/JT deposit and redemption entry points that call into the accountant to perform the accounting. Two vault tranche contracts, RoycoSeniorTranche and RoycoJuniorTranche, extend a shared RoycoVaultTranche base and serve as the LP tokens for investing into the respective tranche. Each Tranche implements an EIP-4626 vault. Yield allocation between tranches is delegated to a pluggable Yield Distribution Model (YDM), with both static-curve and adaptive-curve implementations in scope.

Core Invariants

The protocol's central invariant is NAV conservation: the sum of raw assets held by both tranches must always equal the sum of effective claims against them (INV01). A violation in either direction, that is value created from nothing or silently destroyed, breaks the solvency guarantee that the entire tranche structure depends on.

Several supporting guarantees reinforce this core invariant. Coverage configuration (CNF01, CNF02, CNF03) ensures the coverage parameters for computing utilization are always set to sensible values. Market state transitions between PERPETUAL and FIXED_TERM modes must follow a strict lifecycle (INV02, INV03, INV05, INV06) so that in fixed term mode ST holders cannot exit, when there are outstanding losses absorbed by JT. To protect ST holders, there is always a time limit for fixed term mode and when the protocol becomes distressed (JT tokens can no longer cover the losses), the perpetual mode is entered immediately. If the system becomes distressed the impermanent losses of JT are erased (made permanent) ensuring that ST and JT cannot both hold impermanent losses. Protocol fee minting must not dilute LP positions beyond the configured rate (PRE03, POST01) and must not occur at all during FIXED_TERM recovery periods. The senior tranche's NAV per share may only decrease when the junior tranche has been fully depleted (KER06), ensuring that the JT buffer is exhausted before

ST holders bear any loss. Pre-operation sync correctness guarantees that yield is only distributed when no impermanent loss is outstanding and the market is in PERPETUAL state (PRE01), that fixed-term periods always carry a valid end timestamp (PRE02), that new ST losses can only arise once JT effective NAV reaches zero (PRE04), and that JT cannot accrue new value while ST is in a loss position (PRE05). Post-operation sync correctness ensures that effective NAV changes mirror raw NAV changes without cross-tranche leakage (POST02), that only JT deposits can increase JT effective NAV (POST03), and that impermanent loss decreases proportionally on redemption for both tranches (POST04, POST05).

Trust Boundaries and Security Assumptions

Trust boundaries in the system separate several distinct roles. Administrative roles, accessed behind a two-day timelock, control coverage parameters, liquidation thresholds, fee rates, and the fixed-term duration. The YDM is a critical trust-boundary surface: since it is pluggable, a misconfigured or malicious YDM could return yield shares exceeding 100%, producing negative allocations to the opposite tranche. Kernel implementations are trusted to call the accountant's post-operation sync and coverage enforcement on every code path — a missing call in a new kernel variant would silently bypass coverage checks. Transfer agent roles can force-move tranche shares for compliance purposes, which creates a secondary trust assumption around that privilege. LP whitelisting, when enabled, must be enforced consistently across mints, burns, and transfers to prevent unauthorized exposure (AUTH01).

Attack Vectors

The attack vectors considered during this engagement span four broad categories. Arithmetic and accounting risks include overflow, unsafe casts, rounding accumulation, and dust-tolerance exploitation. State-consistency risks cover concurrent syncs, reentrant kernel calls, and stale NAV during fee minting. Economic manipulation encompasses adversarial timing of deposits or redemptions around utilization snapshots, front-running accounting syncs to exploit fee-ordering dependencies, round-trip extraction through deposit-redeem or redeem-deposit sequences and forcing liquidation during transient market dislocations. Access-control bypasses address ERC-20 permit-based gasless approvals circumventing front-end controls and incomplete restriction enforcement across mint, burn, and transfer paths.

Verification Approach

Formal verification of the **RoycoAccountant** covered NAV conservation (INV01), coverage constraint enforcement (CNF01–CNF03), impermanent loss exclusivity and lifecycle correctness (INV02, INV03, INV05, INV06), market state transition integrity (PRE01–PRE05), protocol fee bounds (PRE03, POST01), and post-operation accounting correctness for all key state-modifying functions (POST02–POST05, KER06).

Formal verification of the **RoycoKernel** and its instantiations focused on four areas. First, access-control guards: in FIXED_TERM mode, stRedeem and jtDeposit must revert (KER01), and new ST deposits must revert while stImpermanentLoss is positive (KER02). Second, economic soundness of the exchange rate: round-trip properties requiring that an immediate redeem-then-deposit or deposit-then-redeem cannot be profitable (KER03, KER04), and that splitting a deposit or redeem into two calls does not yield more shares or assets than a single combined call (KER05). Third, share-value integrity: the NAV-per-share ratio for both ST and JT must not decrease across any operation (KER07, KER08), and the ST NAV per share may only decline once the JT buffer is fully depleted (KER06). Fourth, correctness of the protocol's bound functions: maxDeposit returns a safe upper bound that does not push utilization above 100% (KER10), and maxRedeem returns a bound below which redemption preserves coverage (KER09). Utilization enforcement was verified separately: stDeposit and jtRedeem must leave utilization within the coverage limit (UTI03), and stRedeem must not worsen utilization (UTI02). The kernel was also verified to always call postOpSyncTrancheAccounting with arguments consistent with the operation type (KER11). KER03, KER04, KER05, KER07, KER08, KER09, and the jtDeposit direction of UTI02 are violated due to rounding issues documented in L-01, L-03, and L-04.

Formal verification of the **RoycoVaultTranche** covered three properties. Authorization enforcement (AUTH01) confirms that every non-view operation on the tranche contracts and the kernel requires a successful canCall authorization check, that is, unauthorized calls must revert. Asset accounting integrity (KER12) confirms that the kernel's actual token balance always equals the sum of stOwnedYieldBearingAssets and jtOwnedYieldBearingAssets, verified separately for the identical-asset and separate-asset kernel variants. Supply integrity (TRA01) confirms that the ERC-20 totalSupply of both JT and ST always equals the sum of all individual holder balances, ensuring no shares are minted or burned outside of proper accounting.

The full property list and per-module assumption details are provided in the Formal Verification section of this report.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	4		
Informational	2		
Total	6		

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
L-01	Deposit can mint 1 NAV unit more shares due to rounding	Low	Not Fixed
L-02	Uncheck cast when converting NAV to signed	Low	Not Fixed
L-03	Redeem can lower the share/asset ratio due to rounding	Low	Not Fixed
L-04	Depositing Junior Tokens can slightly increase computed utilization	Low	Not Fixed
I-01	Some functions do not update the market state	Informational	Not Fixed
I-02	Royco uses 32-bit time stamps	Informational	Not Fixed

Low Severity Issues

L-01 Deposit can mint 1 NAV unit more shares due to rounding

Severity: Low	Impact: Low to Medium	Likelihood: Low
Files: RoycoKernel.sol	Status: Not Fixed	Violated Property: Round in favor of the Protocol

Description: The deposit function grants shares according to the increment in effective NAV in the post state:

```
JavaScript
jtPostDepositNAV = (_postOpSyncTrancheAccounting(Operation.JT_DEPOSIT,
ZERO_NAV_UNITS)).jtEffectiveNAV;
navToMintSharesAt = state.jtEffectiveNAV;
valueAllocated = (jtPostDepositNAV - navToMintSharesAt);
```

Here jtPostDepositNAV is state.jtEffectiveNAV plus the increment in raw NAV. So the increment in raw NAV is what determines valueAllocated. The problem is that this ignores rounding. The raw NAV is computed by this function:

```
JavaScript
   function _getJuniorTrancheRawNAV() internal view virtual returns (NAV_UNIT jtRawNAV) {
        // Get the yield bearing assets owned by JT and convert them to NAV units via the
configured quoter
        return
jtConvertTrancheUnitsToNAVUnits(_getRoycoKernelStorage().jtOwnedYieldBearingAssets);
    }
    function jtConvertTrancheUnitsToNAVUnits(TRANCHE_UNIT _jtAssets) public view
override(RoycoKernel) returns (NAV_UNIT nav) {
        return _convertTrancheUnitsToNAVUnits(_jtAssets);
    }
    function _convertTrancheUnitsToNAVUnits(TRANCHE_UNIT _assets) internal view returns
(NAV_UNIT) {
```

```
        return
toNAVUnits(toUint256(_assets.mulDiv(_getCachedTrancheUnitToNAVUnitConversionRateWAD(),
TRANCHE_UNIT_SCALE_FACTOR, Math.Rounding.Floor)));
    }
```

Note that the mulDiv rounds down for the pre and post state.
If the raw NAV before the deposit is close to the next integer, e.g. 100.9995, and 5.001 NAV units worth of tranche tokens are deposited, then the new raw nav is 106.0005. Since the code rounds down in both cases, the valueAllocated will be set to 106-100 = 6. So the depositor unfairly gains 1 NAV unit worth of tokens.

Exploit Scenario: For this bug to be exploitable, the value of a tranche share should be smaller than 1 NAV (so that the rounding error due to share rounding does not eat the 1 NAV profit). This can happen if the tranche realized losses in the past. Furthermore, 1 NAV unit must have enough value to pay for gas cost (a few thousand units of gas worth). The latter is quite unlikely, because NAV always has 18 digits of precision.

If these conditions are satisfied, then repeatedly depositing and redeeming can be used to drain the protocol by tweaking the values such that each deposit results in 1 NAV unit profit.

Recommendations: Instead of using the increment in raw NAV, the function `jtDeposit` should use `convertTrancheUnitsToNAVUnits` directly on the supplied assets, which rounds the result down. Similarly for `stDeposit`.

Customer's response: {Customer feedback}

Fix Review: {Comments about the fix}

L-02 Uncheck cast when converting NAV to signed

Severity: Low	Impact: Medium	Likelihood: Very Low
Files: RoycoKernel.sol	Status: Not Fixed	Violated Property: Accounting Invariants

Description: The units library defines a function to convert NAVUNIT into signed int256. This function uses an unchecked cast:

```
JavaScript
function toInt256(NAV_UNIT _units) pure returns (int256) {
    return int256(NAV_UNIT.unwrap(_units));
}
```

For very large values of NAV this can return a negative value.

Exploit Scenario: This is not naturally exploitable. It would require a token with an unreasonable total supply to be enabled in the protocol or a broken oracle giving unreasonable large prices.

Recommendations: Check the result of the cast and revert for negative values. This would freeze the contract instead of violating important invariants:

```
JavaScript
  function toInt256(NAV_UNIT _units) pure returns (int256) {
-     return int256(NAV_UNIT.unwrap(_units));
+     int256 assets = int256(NAV_UNIT.unwrap(_units));
+     // check for signed overflow
+     require(assets >= 0, ASSETS_MUST_BE_NON_NEGATIVE());
+     return assets;
  }
```

We used this fix for verifying the properties listed in the later sections. Without the fix several properties are violated.

Customer's response: {Customer feedback}

Fix Review: {Comments about the fix}

L-03 Redeem can lower the share/asset ratio due to rounding

Severity: Low	Impact: Low	Likelihood: Low
Files: RoycoKernel.sol	Status: Not Fixed	Violated Property: Round in favor of the Protocol

Description: If you redeem twice the same number of shares, the expectation is to get the same number of tokens. Because of rounding the exact value can change, but you expect the first user to pay for the rounding and the second user to get the same or more tokens. But it can be that the second user gets less tokens on redeem. The problem is that a total NAV value is computed from the assets and involves rounding. If the rounding error before a redeem is smaller than the rounding error after a redeem, the value of one share can decrease in NAV. This is only probable if a single asset has a value of less than a single NAV unit and there are only a few NAV units in the contract.

Scenario: 100 Assets = 1 NAV wei, Contract has 500 Assets and 20 shares (including virtual). The NAV is exactly 5 wei. The first user who withdraws gets 25 Assets per share, so for redeeming 5 shares they get 125 Assets. However this reduces the NAV to 3 wei, because it is rounded down. The next user now gets a ratio of 300/15=20 Assets/Share, so they only receive 100 Assets when redeeming 5 shares.

Recommendations: One way to ensure fairness for the second user is to be more unfair to the first by rounding twice: first compute his redeemed NAV rounding it down, then use the asset price to determine the number of assets rounding down again. Since the difference is at most a single NAV wei it is also reasonable to ignore this problem.

Customer's response: {Customer feedback}

Fix Review: {Comments about the fix}

L-04 Depositing Junior Tokens can slightly increase computed utilization

Severity: Low	Impact: Low	Likelihood: Low
Files: UtilsLib.sol	Status: Not Fixed	Violated Property: jtDeposit decreases utilization

Description: The utilization is computed as

$$\text{utilization} = \frac{cov \cdot (stRawNAV + \beta \cdot jtRawNAV)}{jtEffectiveNAV} \text{ where } cov \cdot \beta \leq 1$$

The deposit function increases jtRawNAV and jtEffectiveNAV by the same amount, which decreases utilization because of the side condition $cov \cdot \beta \leq 1$. However, the code itself does more rounding and rounds $\beta \cdot jtRawNAV$ to the next largest integer:

```
JavaScript
        utilization = _coverageWAD.mulDiv((_stRawNAV + _jtRawNAV.mulDiv(_betaWAD, WAD,
Math.Rounding.Ceil)), _jtEffectiveNAV, Math.Rounding.Ceil);
```

Under certain circumstances this additional rounding can increase the numerator in the fraction computing the utilization by an additional NAV, which increases the utilization beyond what it was before the call. It's even possible that a utilization of < 1 changes to a utilization of > 1.
Note that the real utilization decreases, only the computed utilization increases. This issue cannot be exploited by multiple small deposits as this is just a "display" artifact of showing a slightly too high utilization.

The same rounding error can also cause jtRedeem to revert when called with maxJTRedeem. The actual utilization is smaller than 1 in that case, but because of rounding for very small total nav values (the counterexample uses 4 NAV wei in the contract in total) the rounding errors can cause the utilization to be greater than 1.

Scenario: Before deposit: stRawNAV = 1, jtRawNAV = 0, jtEffectiveNAV = 1, β = 1.2, cov = 0.7, utilization is 1*0.7 / 1 = 0.7, well below 1. After depositing of 1 jtNAV, the term $\beta \cdot jtRawNAV$ is rounded to 2 (1.2 rounded up) and utilization is 3*0.7/2 =1.05.

Recommendations: This is an edge case error occurring when only a few NAV are deposited into the contract. The real utilization decreases, but because of rounding, the computed utilization is too large. This should normally not be a problem, but to be extra sure, the liquidationUtilization should not be too close to 1.0.

Customer's response: {Customer feedback}

Fix Review: {Comments about the fix}

Informational Issues

I-01. Some functions do not update the market state.

Description: Some functions change some values that would need the market state to be recomputed, but they don't change the market state. This is only a view problem, since the functions that need the market state such as the redeem and deposit functions will recompute it before applying the operation.
We found the following issues:
`set*TrancheDustTolerance` does not check whether the jtImpermanentLoss becomes larger than dust by the change. Then the market is not immediately moved to FIXED_TERM.
`jtRedeem` decreases the jtImpermanentLoss but doesn't check if it becomes 0 (e.g., when all junior tranche is redeemed). Then the market is not immediately moved to PERPETUAL again.

Recommendation: It's not a big issue, but it would be cleaner to set the market state in these functions correctly.

Customer's response: {Customer's response}

Fix Review: {Comments about the fix}

I-02. Royco uses 32-bit time stamps

Description: RoycoAccountant stores three timestamps as `uint32` in its accounting storage:

```
JavaScript
uint32 lastAccrualTimestamp;
uint32 lastDistributionTimestamp;
uint32 fixedTermEndTimestamp;
```

The `uint32` type can hold values up to 2^32 - 1 = 4,294,967,295, which as a Unix timestamp corresponds to February 7, 2106. After that date, `uint32(block.timestamp)` silently truncates the upper bits, storing a corrupted value. Elapsed-time calculations then return a vastly inflated result:

```
JavaScript
// After year 2106, block.timestamp exceeds uint32 max
$.lastAccrualTimestamp = uint32(block.timestamp); // wraps to small value
uint256 elapsed = block.timestamp - $.lastAccrualTimestamp; // enormous
```

Recommendation: Replace `uint32` with `uint48` for all three timestamp fields. The `uint48` type supports timestamps up to the year 8,921,556 (effectively unlimited) while still packing efficiently into storage slots.

Customer's response: {Customer's response}

Fix Review: {Comments about the fix}

Formal Verification

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

The following assumptions are shared across all properties and rules of the RoycoAccountant, the RoycoKernel and the RoycoVaultTranche modules.

1. The maximum supported timestamp is the year 2106, see I-02.
2. We applied the recommended fix of L-02 to avoid reporting violations due to this bug.
3. Calls to upgradeToAndCall are not considered when verifying global properties — it can violate all properties.
4. Calls to initialize are not considered for some properties - initialize cannot be called after initialization, but before it may break some properties
5. stNAVDustTolerance is 0 in PRE01 to simplify the fee/loss relationship
6. The jtYieldShare function is summarized abstractly; the verification is correct for any function whose return value is bounded to be less than WAD (i.e., JT never receives 100% or more of yield in a single distribution). We assume that it has no side-effects or reentrant calls.

Detailed Properties

RoycoAccountant

Module Assumptions

- For CNF03, uninitialized contracts (where `_initialized != max_uint64`) are excluded, as coverage may not yet be set.
- INV02 excludes the `initialize` and `upgradeAndCall` functions from verification.
- PRE01 assumes `stNAVDustTolerance = 0` and `jtNAVDustTolerance = 0` to avoid failures caused by dust-level `jtImpermanentLoss`.
- All other verified properties for this module require no additional assumptions.

Module Properties

CNF01: liquidationUtilizationWAD > WAD

Status: Verified

Rule Name	Status	Description	Link to rule report
liquidationGreaterThanOne	Verified	*The liquidation threshold must exceed 100% utilization to provide a meaningful safe zone between full coverage and forced liquidation.*	*rule report*

CNF02: coverageWAD * betaWAD < WAD

Status: Verified

Rule Name	Status	Description	Link to rule report
coverageBetaLessThanOne	Verified	*The product of coverage and beta must be below 1 to ensure the utilization denominator remains positive and the coverage formula yields valid results.*	*rule report*

CNF03: MIN_COVERAGE ≤ coverage ≤ MAX_COVERAGE

Status: Verified	

Rule Name	Status	Description	Link to rule report
coverageGreaterEqualMin	Verified	*The coverage parameter has a minimum of 1% to prevent degenerate configurations that would make the utilization formula meaningless.*	*rule report*
coverageLessEqualMax	Verified	*The coverage parameter has a maximum just below 100% to ensure the protocol always retains some buffer above the liquidation threshold.*	*rule report*

INV01: jtEffectiveNAV + stEffectiveNAV = jtRawNAV + stRawNAV

Status: Verified	

Rule Name	Status	Description	Link to rule report
sumEffectiveEqualsRaw	Verified	*The total effective NAV equals the total raw NAV at all times: impermanent losses are transferred between tranches, not created or destroyed.* *Note: When checking this property, we removed the reverts in RoycoAccountant for NAV_CONSERVATION_VIOLATION(). Thus, we proved that these reverts are redundant.*	*rule report*

INV02: jtImpermanentLoss > dust ⇒ marketstate = FIXEDTERM

Status: Violated	See I-01

Rule Name	Status	Description	Link to rule report
jtLossImpliesFixed	Violated by	*If jtImpermanentLoss exceeds the dust tolerance, the market must be in FIXED_TERM state. JT losses*	*rule report*

Term	setDustTolerance, see I-01	*only arise when a fixed-term market settles below par.*	

INV03: jtImpermanentLoss = 0 ⇒ marketstate = PERPETUAL			
Status: Violated		See I-01	
Rule Name	Status	Description	Link to rule report
noJTLossImpliesPerpetual	Violated by postOpSyncTrancheAccounting (jtRedeem), see I-01	*When jtImpermanentLoss = 0, the market must be in PERPETUAL state; there is no ongoing fixed-term settlement with unrecovered losses.*	*rule report*

INV04: stImpermanentLoss > 0 ⇒ jtEffectiveNAV=0

This was a wrong property (violated). Replaced by PRE04 and PRE05

INV05: stImpermanentLoss > 0 ⇒ marketstate = PERPETUAL (distressed state)			
Status: Verified			
Rule Name	Status	Description	Link to rule report
stLossImpliesPerpetual	Verified	*When stImpermanentLoss > 0, the market is in a distressed PERPETUAL state where JT has been fully depleted and ST bears residual losses.*	*rule report*
stLossImpliesNoJTLoss	Verified	*ST and JT losses cannot coexist: JT loss occurs during FIXED_TERM settlement, while ST loss occurs in the PERPETUAL distressed state after JT is depleted.*	*rule report*

INV06: fixedTermDurationSeconds = 0 ⇒ marketstate = PERPETUAL (no fixedterm allowed)

Status: Verified

Rule Name	Status	Description	Link to rule report
termDurationZeroAlwaysPerpetual	Verified	*When the fixed-term duration is configured as 0, no fixed-term transitions are allowed and the market must always remain in PERPETUAL mode.*	rule report

KERO6: NAV for one ST share should only decrease if stImpermanentLoss increases. In that case JT tokens have zero value

Status: Verified

Rule Name	Status	Description	Link to rule report
StNAVIncreasesUnlessJTIsZero	Verified	*The ST NAV can only decline if JT has been fully depleted (jtEffectiveNAV = 0), meaning losses have exhausted the JT buffer and are now hitting ST.*	rule report

POST01: No fees distributed

Status: Verified

Rule Name	Status	Description	Link to rule report
postSyncNoFees	Verified	*The post-operation sync only updates NAV accounting; protocol fees are exclusively distributed through preOpSync.*	rule report

POST02: Share value should not change, except for the redemption bonus when liquidity > 0

Status: Verified

Rule Name	Status	Description	Link to rule report
postSyncRawChangeDirectlyReflected	Verified	*The total effective NAV change always equals the total raw NAV change. Cross-tranche effects follow per-operation rules: ST_DEPOSIT and JT_DEPOSIT do not affect the other tranche's effective NAV (except ST_REDEEM which transfers the self-liquidation bonus to JT).*	*rule report*

POST03: jtEffectiveNAV should only increase by jtDeposit

Status: Verified

Rule Name	Status	Description	Link to rule report
postSyncOnlyJTDepositIncreasesJTEffective	Verified	*The only operation that may increase jtEffectiveNAV is JT_DEPOSIT; all other operations (ST_DEPOSIT, ST_REDEEM, JT_REDEEM) must not increase jtEffectiveNAV.*	*rule report*

POST04: on redeem, stImpermanentLoss should decrease proportionately

Status: Verified

Rule Name	Status	Description	Link to rule report
postSyncSTImpermanentLossProportionally	Verified	*For *_REDEEM operations, stImpermanentLoss decreases in proportion to the reduction in stEffectiveNAV, ensuring the loss-per-NAV ratio is preserved (up to rounding).*	*rule report*

POST05: on redeem, jtImpermanentLoss should decrease proportionately.

Status: Verified

Rule Name	Status	Description	Link to rule report
postSyncJTImpermanentLossProportionally	Verified	*For *_REDEEM operations, jtImpermanentLoss decreases in proportion to the reduction in jtEffectiveNAV, ensuring the loss-per-NAV ratio is preserved (up to rounding).*	rule report

PRE01: yieldDistributed ⇒ stImpermanentLoss = jtImpermanentLoss = 0 && marketstate = PERPETUAL

Status: Verified

Rule Name	Status	Description	Link to rule report
preSyncNoYieldMeansNoFee	Verified	*If either protocol fee is non-zero (yield was distributed), then both stImpermanentLoss and jtImpermanentLoss must be zero and the market must be in PERPETUAL state.*	rule report

PRE02: marketstate = FIXEDTERM ⇒ fixedTermEndTimeStamp > 0

Status: Verified

Rule Name	Status	Description	Link to rule report
fixedTermIsBounded	Verified	*A fixed-term period without a deadline is invalid; when in FIXED_TERM state the end timestamp must be non-zero.*	rule report
preSyncFixedTermHasTimestamp	Verified	*When preOpSync returns a FIXED_TERM market state, the end timestamp must be non-zero — a fixed-term period without a deadline is invalid.*	rule report

PRE03: marketState = FIXEDTERM ⇒ stProtocolFeeAccrued = 0 or jtProtocolFeeAccrued = 0

Status: Verified

Rule Name	Status	Description	Link to rule report
preSyncFixedTermNoFee	Verified	*When preOpSync returns a FIXED_TERM market state, at least one of stProtocolFeeAccrued and jtProtocolFeeAccrued must be zero; fees are only distributed when a market makes profit and in fixed_term the profits are used to repay impermanent losses.*	*rule report*

PRE04: if stImpermanentLoss=0 before the call, then after the call stImpermanentLoss > 0 ⇒ jtEffectiveNAV = 0

Status: Verified

Rule Name	Status	Description	Link to rule report
preSyncNewStLossImpliesJTEffectiveZero	Verified	*If stImpermanentLoss was zero before the call but becomes positive after preOpSync, then jtEffectiveNAV must be zero — JT must be fully depleted before ST can take losses.*	*rule report*

PRE05: if stImpermanentLoss >0 then jtEffectiveNAV doesn't increase

Status: Verified

Rule Name	Status	Description	Link to rule report
preSyncStLossImpliesJTEffectiveCannotIncrease	Verified	*If stImpermanentLoss > 0 before preOpSync, then jtEffectiveNAV must not increase after the call — new yield cannot benefit JT while ST is in a loss position.*	*rule report*

RoycoKernel

Module Assumptions

- The oracle price (getTrancheUnitToNAVUnitConversionRateWAD) is modeled as a constant within each rule execution, that is, price changes across a single transaction are not considered.
- Where rules require price consistency, e.g. KER07 and KER08, the stored raw NAV values are assumed to already be synced to the current oracle price.

safeTransfer and safeTransferFrom are modeled as NONDET; the verification assumes tokens are non-reentrant.

Module Properties

KER01: marketstate = FIXEDTERM ⇒ stRedeem and jtDeposit must revert

Status: Verified	*marketstate = FIXED_TERM implies stRedeem and jtDeposit must revert*

Rule Name	Status	Description	Link to rule report
stRedeemRevertsInFixedTerm	Verified	*In a fixed-term market, ST holders cannot redeem; the call must revert if the market state is FIXED_TERM.*	*rule report*
jtDepositRevertsInFixedTerm	Verified	*In a fixed-term market, new JT deposits are not allowed; the call must revert if the market state is FIXED_TERM.*	*rule report*

KER02: stImpermanentLoss > 0 ⇒ stDeposit must revert

Status: Verified	*stImpermanentLoss > 0 implies stDeposit must revert*

Rule Name	Status	Description	Link to rule report
stDepositRevertsWithImpermanentLo	Verified	*When the ST tranche is in a distressed state (stImpermanentLoss > 0), new ST deposits must revert to prevent dilution of the loss.*	*rule report*

ss			

KERO3: redeem immediately followed by deposit should not be profitable; should only lose a few tokens due to rounding.

Status: Violated	*redeem immediately followed by deposit should not be profitable (at most rounding losses)*

Rule Name	Status	Description	Link to rule report
redeemDepositJunior	Violated, see L-01, L-03	*Redeeming JT shares and immediately redepositing the received assets must yield at most the original share amount.*	*rule report*
redeemDepositSenior	Violated, see L-01, L-03	*Redeeming ST shares and immediately redepositing the received assets must yield at most the original share amount.*	*rule report*

KERO4: deposit immediately followed by redeem should not be profitable (if already invested).

Status: Violated	*deposit immediately followed by redeem should not be profitable (if already invested)*

Rule Name	Status	Description	Link to rule report
depositRedeemJunior	Violated, see L-01, L-03	*Depositing into the JT tranche and immediately redeeming the received shares must yield at most the deposited amount.*	*rule report*
depositRedeemSenior	Violated, see L-01, L-03	*Depositing into the ST tranche and immediately redeeming the received shares must yield at most the deposited amount.*	*rule report*

KERO5: Multiple deposits or multiple redeems immediately after each other should get the same exchange rate; exception: selfLiquidation bonus is only paid until the utilization recovers.

Status: Violated	*multiple deposits or redeems in sequence should get the same exchange rate; splitting should not be profitable*

Rule Name	Status	Description	Link to rule report
depositSplitSenior	Violated, see L-01	*Depositing amount1 and amount2 separately into ST must yield no more shares than depositing amount1+amount2 in a single call.*	*rule report*
redeemSplitSenior	Violated, see L-03	*Redeeming amount1 and amount2 separately from ST must yield no more assets than redeeming amount1+amount2 in a single call.*	*rule report*
depositSameJunior	Violated, see L-01	*Two consecutive JT deposits of the same amount must receive the same number of shares, confirming that a deposit does not change the exchange rate.*	*rule report*
redeemSameJunior	Violated, see L-03	*Two consecutive JT redeems of the same share amount must receive identical assets, confirming that a redeem does not change the exchange rate.*	*rule report*
depositSameSenior	Violated, see L-01	*Two consecutive ST deposits of the same amount must receive the same number of shares, confirming that a deposit does not change the exchange rate.*	*rule report*

KERO7: One ST share should be priced at stEffectiveNAV / SeniorTranche.totalSupply Minting/burning shares should not decrease this ratio.

Status: Violated	*One ST share value (stEffectiveNAV / stTotalSupply) must not decrease across any operation*

Rule Name	Status	Description	Link to rule report
stTokenValueDoesNotWorsen	Violated, see L-03	*The ratio stEffectiveNAV / stTotalSupply must not decrease after any operation (excluding upgradeToAndCall and mintProtocolFeeShares).*	*rule report*

KER08: One JT share should be priced at jtEffectiveNAV / JuniorTranche.totalSupply Minting/burning shares should not decrease this ratio.

Status: Violated	*One JT share value (jtEffectiveNAV / jtTotalSupply) must not decrease across any operation (except when self-liquidation bonus applies)*

Rule Name	Status	Description	Link to rule report
jtTokenValueDoesNotWorsen	Violated, see L-03	*The ratio jtEffectiveNAV / jtTotalSupply must not decrease after any operation (excluding upgradeToAndCall and mintProtocolFeeShares). The self-liquidation bonus is excluded when utilization is below 100%.*	*rule report*

KER09: redeem(maxRedeem()) should not revert (unless blacklisted, paused, etc)

Status: Violated	*maxJTWithdrawalGivenCoverage returns a correct upper bound.*

Rule Name	Status	Description	Link to rule report
maxJtRedeemCorrect	Violated, see L-04	*Redeeming any amount strictly less than the maximum returned by maxJTWithdrawalGivenCoverage must leave utilization at or below 100% (WAD); the function must not return an over-estimate that would allow a violating redeem.*	*rule report*

KER10: deposit(maxDeposit()) should not revert (unless blacklisted, paused, etc)

Status: Verified	*maxSTDepositGivenCoverage returns a correct upper bound: depositing up to the maximum must not push utilization above 100%*

Rule Name	Status	Description	Link to rule report
maxStDepositCorrect	Verified	*Depositing any amount up to maxSTDepositGivenCoverage must leave utilization at or below 100% (WAD); the function must not return an over-estimate that would allow a violating*	*rule report*

		deposit.	

KER11: The kernel calls postOpSyncTrancheAccounting with arguments consistent with the operation type

Status: Verified	*The kernel calls postOpSyncTrancheAccounting with arguments consistent with the operation type: deposits increase NAV, redeems decrease NAV, no cross-tranche NAV changes on deposit, no self-liquidation bonus on deposit or JT_REDEEM*

Rule Name	Status	Description	Link to rule report
checkPostOpUsage	Verified	*For every kernel operation, the arguments passed to postOpSyncTrancheAccounting must be consistent with the operation type: deposits increase NAV (not decrease), redeems decrease NAV (not increase), deposits carry no self-liquidation bonus, JT operations do not cross-affect ST NAV.*	*rule report*

UTI02: stRedeem and jtDeposit should never increase utilization

Status: Violated	*stRedeem and jtDeposit always decrease or preserve utilization (coverage requirement remains satisfied)*

Rule Name	Status	Description	Link to rule report
jtDepositPreservesUtilization	Violated, see L-04	*A JT deposit adds JT-side coverage and must not worsen (increase) utilization; if coverage was satisfied before, it remains satisfied after.*	*rule report*
stRedeemPreservesUtilization	Verified	*An ST redeem removes ST-side exposure and must not worsen utilization; if coverage was satisfied before, it remains satisfied after.*	*rule report*

UTI03: stDeposit and jtRedeem should respect utilization (unless they were seized).

Status: Verified	*stDeposit and jtRedeem must not violate the coverage requirement (utilization must remain satisfied after)*

Rule Name	Status	Description	Link to rule report
stDepositEnsuresUtilization	Verified	*After an ST deposit, the coverage requirement (utilization ≤ 100%) must be satisfied; ST deposits increase the pool that needs to be covered but must not push utilization over the limit.*	*rule report*
jtRedeemEnsuresUtilization	Verified	*After a JT redeem, the coverage requirement must be satisfied; the implementation must revert if redeeming JT would push utilization over the limit.*	*rule report*

RoycoVaultTranche

Module Assumptions

- ERC-20 tokens are standard, no fee taking, no reentrancy, no rebasing.

Module Properties

AUTH01: All operations except transfer in RoycoVaultTranche are protected

Status: Verified	*All non-view operations on RoycoVaultTranche (and the kernel) require canCall authorization to succeed*

Rule Name	Status	Description	Link to rule report
checkRestricted	Verified	*Every non-reverting call to any tranche or kernel function must have been authorized via canCall(msg.sender, target, selector); unauthorized calls must revert.*	*rule report*

KER12: kernel.stOwnedYieldBearingAssets + kernel.jtOwnedYieldBearingAssets equals the kernel's actual token balance

Status: Verified	*kernel.stOwnedYieldBearingAssets + kernel.jtOwnedYieldBearingAssets equals the kernel's actual token balance (for identical-asset and separate-asset variants)*

Rule Name	Status	Description	Link to rule report
kernelTokenBalances	Verified	*When ST and JT share the same asset, the kernel's actual token balance must equal stOwnedYieldBearingAssets + jtOwnedYieldBearingAssets.*	*rule report*
kernelTokenStBalances	Verified	*When ST and JT use different assets, the kernel's ST token balance must equal stOwnedYieldBearingAssets independently.*	*rule report*
kernelTokenJtBalances	Verified	*When ST and JT use different assets, the kernel's JT token balance must equal jtOwnedYieldBearingAssets independently.*	*rule report*

TRA01: JT and ST totalSupply always equals the sum of all holder balances	
Status: Verified	*JT and ST totalSupply always equals the sum of all holder balances*

Rule Name	Status	Description	Link to rule report
jtTotalSupplyIsSumOfBalances	Verified	*The JT ERC20 totalSupply must always equal the sum of all individual balances tracked by the ghost, confirming no shares are minted or burned outside of proper accounting.*	*rule report*
stTotalSupplyIsSumOfBalances	Verified	*The ST ERC20 totalSupply must always equal the sum of all individual balances tracked by the ghost, confirming no shares are minted or burned outside of proper accounting.*	*rule report*

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.